

计算机与信息工程学院实验报告

姓名	——	学号	
专业		年级	
课程	操作系统	主讲教师	刘扬
实验时间（年月日时）	2025 年 9 月 25 日 10 时		
实验地点	计算机大楼 313	辅导教师	刘扬
实验题目	进程管理		
实验目的	（1）加深对进程概念的理解，明确进程和程序的区别。 （2）进一步认识并发执行的实质。 （3）分析进程竞争资源现象，学习解决进程互斥的方法。 （4）了解 Linux 系统中进程通信的基本原理。		
实验环境（硬件和软件）	VMware+kali		

一、OS 课程实验

实验内容：

- 1、进程的创建：编写一段程序，使用系统调用 `fork()` 创建两个子进程。当此程序运行时，在系统中有一个父进程和两个子进程活动。让每一个进程在屏幕上显示一个字符；父进程显示字符“a”，子进程分别显示字符“b”和“c”。试观察记录屏幕上的显示结果，并分析原因。
- 2、进程的控制：修改已编写好的程序，将每个程序的输出由单个字符改为一句话，再观察程序执行时屏幕上出现的现象，并分析其原因。如果在程序中使用系统调用 `lockf()` 来给每个程序加锁，可以实现进程之间的互斥，观察并分析出现的现象。
- 3、软中断通信：（1）编制一段程序，使用系统调用 `fork()` 创建两个子进程，再用系统调用 `signal()` 让父进程捕捉键盘上来的中断信号（即按 `ctrl+c` 键），当捕捉到中断信号后，父进程用系统调用 `kill()` 向两个子进程发出信号，子进程捕捉到信号后，分别输出下列信息后终止：
child process1 is killed by parent!
child process2 is killed by parent!
父进程等待两个子进程终止后，输出以下信息后终止：

parent process is killed!

(2) 在上面的任务 1 中, 增加语句 `signal(SIGINT,SIG_IGN)` 和语句 `signal(SIGQUIT,SIG_IGN)`, 观察执行结果, 并分析原因。这里, `signal(SIGINT,SIG_IGN)`和 `signal(SIGQUIT,SIG_IGN)`分别为忽略键信号以及忽略中断信号。

4、进程的管道通信: 编制一段程序, 实现进程的管道通信。使用系统调用 `pipe()` 建立一条管道线。两个子进程 `p1` 和 `p2` 分别向管道个写一句话:

child1 process is sending message!

child2 process is sending message!

而父进程则从管道中读来自两个进程的信息, 显示在屏幕上。

实验步骤:

1、进程的创建:

代码如下

```
$ cat ./fork.c
#include <stdio.h>
#include <unistd.h>
int main(){
    int p1, p2;
    if(p1 = fork()){
        putchar('b');
    }else {
        if(p2 = fork()){
            putchar('c');
        }else {
            putchar('a');
        }
    }
    return 0;
}
```

编译及运行结果

```

(ygkldnherr@kali)-[~/Documents]
$ gcc -o fork.out ./fork.c

(ygkldnherr@kali)-[~/Documents]
$ ./fork.out
bc
a
(ygkldnherr@kali)-[~/Documents]
$ ./fork.out
bca
(ygkldnherr@kali)-[~/Documents]
$ ./fork.out
b
ca
(ygkldnherr@kali)-[~/Documents]
$ ./fork.out
bc
a

```

2、进程的控制

程序 1 代码如下

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(){
    int p1, p2;
    if(p1 = fork()){
        for(int i=0; i<500; i++){
            printf("parent %d\n", i);
        }
        wait(0);
        exit(0);
    }else {
        if(p2 = fork()){
            for(int i=0; i<500; i++){
                printf("son %d\n", i);
            }
            wait(0);
            exit(0);
        }else {
            for(int i=0; i<500; i++){
                printf("grandchild %d\n", i);
            }
            exit(0);
        }
    }
    return 0;
}

```

编译及运行结果

```
(ygkldnherr@kali)-[~/Documents]
$ gcc -o fork1.out ./fork1.c

(ygkldnherr@kali)-[~/Documents]
$ ./fork1.out
parent 0
parent 1
parent 2
parent 3
parent 4
parent 5
parent 6
parent 7
parent 8
parent 9
parent 10
parent 11
parent 12
parent 13
parent 14
```

parent 154	son 45
parent 165	grandchild 0
son 0	parent 213
parent 166	son 46
son 1	parent 214
parent 167	grandchild 1
son 2	son 47
parent 168	parent 215
son 3	grandchild 2
parent 169	son 48
son 4	parent 216
parent 170	grandchild 3
son 5	son 49
parent 171	parent 217
son 6	grandchild 4
parent 172	son 50
son 7	parent 218
parent 173	grandchild 5
son 8	son 51
parent 174	parent 219
son 9	grandchild 6
parent 175	son 52
son 10	parent 220
parent 176	grandchild 7
son 11	son 53
parent 177	parent 221
parent 178	grandchild 8
son 12	son 54
parent 179	parent 222
son 13	grandchild 9
parent 180	son 55
son 14	parent 223
parent 181	grandchild 10
parent 182	son 56
son 15	parent 224
parent 183	grandchild 11
son 16	
parent 184	
son 17	

程序 2 代码如下

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int main(){
    int p1, p2;
    if(p1 = fork()){
        lockf(1, 1, 0);
        for(int i=0; i<500; i++){
            printf("parent %d\n", i);
        }
        lockf(1, 0, 0);
        wait(0);
        exit(0);
    }else {
        if(p2 = fork()){
            lockf(1, 1, 0);
            for(int i=0; i<500; i++){
                printf("son %d\n", i);
            }
            lockf(1, 0, 0);
            wait(0);
            exit(0);
        }else {
            lockf(1, 1, 0);
            for(int i=0; i<500; i++){
                printf("grandchild %d\n", i);
            }
            lockf(1, 0, 0);
            exit(0);
        }
    }
    return 0;
}
```

编译及运行结果

```
(ygkldnherr@kali) - [~/Documents]
$ gcc -o fork2.out ./fork2.c
```

```

(ygkldnherr@kali) - [~/Documents]
$ ./fork2.out
parent 0      son 0      grandchild 461
parent 1      son 1      grandchild 462
parent 2      son 2      grandchild 463
parent 3      son 3      grandchild 464
parent 4      son 4      grandchild 465
parent 5      son 5      grandchild 466
parent 6      son 6      grandchild 467
parent 7      son 7      grandchild 468
parent 8      son 8      grandchild 469
parent 9      son 9      grandchild 470
parent 10     son 10     grandchild 471
parent 11     son 11     grandchild 472
parent 12     son 12     grandchild 473
parent 13     son 13     grandchild 474
parent 14     son 14     grandchild 475
parent 15     son 15     grandchild 476
parent 16     son 16     grandchild 477
parent 17     son 17     grandchild 478
parent 18     son 18     grandchild 479
parent 19     son 19     grandchild 480
parent 20     son 20     grandchild 481
parent 21     son 21     grandchild 482
parent 22     son 22     grandchild 483
parent 23     son 23     grandchild 484
parent 24     son 24     grandchild 485
parent 25     son 25     grandchild 486
parent 26     son 26     grandchild 487
parent 27     son 27     grandchild 488
parent 28     son 28     grandchild 489
parent 29     son 29     grandchild 490
parent 30     son 30     grandchild 491
parent 31     son 31     grandchild 492
parent 32     son 32     grandchild 493
parent 33     son 33     grandchild 494
parent 34     son 34     grandchild 495
parent 35     son 35     grandchild 496
parent 36     son 36     grandchild 497
parent 37     son 37     grandchild 498
parent 38     son 38     grandchild 499

```

- 3、软中断通信
- 任务 1 代码如下：

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
void waiting(), stop(), alarming();
int wait_mark;
void main(){
    int p1, p2;
    if(p1 = fork()){
        if(p2 = fork()){
            wait_mark = 1;
            signal(SIGINT, stop);
            signal(SIGALRM, alarming);
            waiting();

            kill(p1, 16);
            kill(p2, 17);

            wait(0);
            wait(0);
            printf("parent process is killed!\n");
            exit(0);
        }else{
            wait_mark = 1;
            signal(17, stop);
            signal(SIGINT, SIG_IGN);
            while(wait_mark != 0);
            lockf(1, 1, 0);
            printf("child process2 is killed by parent!\n");
            lockf(1, 0, 0);
            exit(0);
        }
    }else{
        wait_mark = 1;
        signal(16, stop);
        signal(SIGINT, SIG_IGN);
        while(wait_mark != 0);
        lockf(1, 1, 0);
        printf("child process1 is killed by parent!\n");
        lockf(1, 0, 0);
        exit(0);
    }
}

void waiting(){
    sleep(5);
    if(wait_mark != 0){
        kill(getpid(), SIGALRM);
    }
}

void alarming(){
    wait_mark = 0;
}

void stop(){
    wait_mark = 0;
}

```

```

fork3.c > main()
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <signal.h>
4  #include <unistd.h>
5  #include <sys/types.h>
6  #include <sys/wait.h>
7  void waiting(), stop(), alarm
8  int wait mark;
9  void main(){
10     int p1, p2;
11     if(p1 = fork()){
12         if(p2 = fork()){
13             wait mark = 1;
14             signal(SIGINT, s
15             signal(SIGALRM,
16             waiting();
17
18             kill(p1, 16);
19             kill(p2, 17);
20
21             wait(0);
22             wait(0);
23             printf("parent
24             exit(0);
25         }else{
26             wait mark = 1;
27             signal(17, stop)
28             signal(SIGINT, S
29             while(wait mark
30             lockf(1, 1, 0);
31             printf("parent p
32             exit(0);
33         }
34     }else{
35         wait mark = 1;
36         signal(16, stop);
37         signal(SIGINT, SIG_
38         while(wait mark !=
39         lockf(1, 1, 0);
40         printf("parent p
41         exit(0);
42     }
43 }
44
45 void waiting(){
46     sleep(5);
47     if(wait mark != 0){
48         kill(getpid(), SIG
49     }
50 }
51
52 void alarming(){
53     wait mark = 0;
54 }
55
56 void stop(){
57     wait mark = 0;
58 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```

(ygklndherr@kali) - [~/Documents/c
$ kill -l
HUP INT QUIT ILL TRAP IOT BUS FPE KI

```

```

(ygklndherr@kali) - [~/Documents/c
$

```

编译及运行结果

```
(ygkldnherr@kali)-[~/Documents/os/exp3]  
$ gcc -o fork3.out ./fork3.c
```

```
(ygkldnherr@kali)-[~/Documents/os/exp3]  
$ ./fork3.out  
^Cchild process1 is killed by parent!  
child process2 is killed by parent!  
parent process is killed!  
  
(ygkldnherr@kali)-[~/Documents/os/exp3]  
$ ./fork3.out  
child process1 is killed by parent!  
child process2 is killed by parent!  
parent process is killed!
```

任务 2 代码如下

```

#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
void IntDelete(), Int1(), Int2();
int wait_mark, pid1, pid2;
int EndFlag = 0;
int pf1 = 0;
int pf2 = 0;
void main(){
    int exitpid;
    if(pid1 = fork()){
        if(pid2 = fork()){
            signal(SIGINT, IntDelete);
            waitpid(-1, &exitpid, 0);
            waitpid(-1, &exitpid, 0);
            printf("parent process is killed\n");
            exit(0);
        }else{
            signal(SIGINT, SIG_IGN);
            signal(17, Int2);
            pause();
        }
    }else{
        signal(SIGINT, SIG_IGN);
        signal(16, Int1);
        pause();
    }
}

void IntDelete(){
    kill(pid1, 16);
    kill(pid2, 17);
}

void Int1(){
    printf("child process 1 is killed!by parent\n");
    exit(0);
}

void Int2(){
    printf("child process 2 is killed!by parent\n");
    exit(0);
}

```

```

fork4.c > main()
12 void main(){
14     if(pid1 = fork1
15         if(pid2 = f
20         exit(0)
21     }else{
22
23         signal(
24         signal(
25         pause()
26     }
27 }else{
28     signal(SIGI
29     signal(16,
30     pause();
31 }
32
33
34
35     kill(pid1, 16);
36     kill(pid2, 17);
37 }
38
39
40     kill(pid2, 17);
41 }
42
43
44 void Int1(){
45     printf("child p
46     exit(0);
47 }
48
49 void Int2(){
50     printf("child p
51     exit(0);
52 }
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99

```

```

(ygkldnherr@kali) - [~/D
HUP INT QUIT ILL TRAP IO
(ygkldnherr@kali) - [~/D
$
• 还原的历史记录
(ygkldnherr@kali) - [~/D
$

```

编译及运行结果

```
(ygkldnherr@kali)-[~/Documents/os/exp3]
$ gcc -o fork4.out ./fork4.c

(ygkldnherr@kali)-[~/Documents/os/exp3]
$ ./fork4.out
^Cchild process 1 is killed!by parent
child process 2 is killed!by parent
parent process is killed
```

4、进程的管道通信

代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
int pid1, pid2;
void main(){
    int fd[2];
    char outpipe[100], inpipe[100];
    pipe(fd);
    while((pid1=fork()) == -1);
    if(pid1 == 0){
        lockf(fd[1], 1, 0);
        sprintf(outpipe, "child 1 process is sending message!");
        write(fd[1], outpipe, 50);
        sleep(5);
        lockf(fd[1], 0, 0);
        exit(0);
    }else{
        while((pid2 = fork()) == -1);
        if(pid2 == 0){
            lockf(fd[1], 1, 0);
            sprintf(outpipe, "child 2 process is sending message!");
            write(fd[1], outpipe, 50);
            sleep(5);
            lockf(fd[1], 0, 0);
            exit(0);
        }else{
            wait(0);
            read(fd[0], inpipe, 50);
            printf("%s\n", inpipe);
            wait(0);
            read(fd[0], inpipe, 50);
            printf("%s\n", inpipe);
            exit(0);
        }
    }
}
```

编译及运行结果

```

(ygkldnherr@kali)-[~/Documents/os/exp3]
$ gcc -o fork5.out ./fork5.c

(ygkldnherr@kali)-[~/Documents/os/exp3]
$ ./fork5.out
child 1 process is sending message!
child 2 process is sending message!

```

实验数据记录:

1、

```

(ygkldnherr@kali)-[~/Documents]
$ gcc -o fork.out ./fork.c

(ygkldnherr@kali)-[~/Documents]
$ ./fork.out
bc
a

(ygkldnherr@kali)-[~/Documents]
$ ./fork.out
bca

(ygkldnherr@kali)-[~/Documents]
$ ./fork.out
b
ca

(ygkldnherr@kali)-[~/Documents]
$ ./fork.out
bc
a

```

2、

```
(ygkldnherr@kali)-[~/Documents]  
$ gcc -o fork1.out ./fork1.c
```

```
(ygkldnherr@kali)-[~/Documents]  
$ ./fork1.out
```

```
parent 0  
parent 1  
parent 2  
parent 3  
parent 4  
parent 5  
parent 6  
parent 7  
parent 8  
parent 9  
parent 10  
parent 11  
parent 12  
parent 13  
parent 14
```

parent 164	son 45
parent 165	grandchild 0
son 0	parent 213
parent 166	son 46
son 1	parent 214
parent 167	grandchild 1
son 2	son 47
parent 168	parent 215
son 3	grandchild 2
parent 169	son 48
son 4	parent 216
parent 170	grandchild 3
son 5	son 49
parent 171	parent 217
son 6	grandchild 4
parent 172	son 50
son 7	parent 218
parent 173	grandchild 5
son 8	son 51
parent 174	parent 219
son 9	grandchild 6
parent 175	son 52
son 10	parent 220
parent 176	grandchild 7
son 11	son 53
parent 177	parent 221
parent 178	grandchild 8
son 12	son 54
parent 179	parent 222
son 13	grandchild 9
parent 180	son 55
son 14	parent 223
parent 181	grandchild 10
parent 182	son 56
son 15	parent 224
parent 183	grandchild 11
son 16	
parent 184	
son 17	

3、

```
(ygkldnherr@kali) - [~/Documents]
$ gcc -o fork2.out ./fork2.c
```

```
(ygkldnherr@kali) - [~/Documents]
$ ./fork2.out
parent 0
parent 1
parent 2
parent 3
parent 4
parent 5
parent 6
parent 7
parent 8
parent 9
parent 10
parent 11
parent 12
parent 13
parent 14
parent 15
parent 16
parent 17
parent 18
parent 19
parent 20
parent 21
parent 22
parent 23
parent 24
parent 25
parent 26
parent 27
parent 28
parent 29
parent 30
parent 31
parent 32
parent 33
parent 34
parent 35
parent 36
parent 37
parent 38
son 0
son 1
son 2
son 3
son 4
son 5
son 6
son 7
son 8
son 9
son 10
son 11
son 12
son 13
son 14
son 15
son 16
son 17
son 18
son 19
son 20
son 21
son 22
son 23
son 24
son 25
son 26
son 27
son 28
son 29
son 30
son 31
son 32
son 33
son 34
son 35
son 36
son 37
son 38
son 39
son 40
son 41
son 42
son 498
son 499
grandchild 0
grandchild 1
grandchild 2
grandchild 3
grandchild 4
grandchild 5
grandchild 6
grandchild 7
grandchild 8
grandchild 9
grandchild 10
grandchild 11
grandchild 12
grandchild 13
grandchild 14
grandchild 15
grandchild 16
grandchild 17
grandchild 18
grandchild 19
grandchild 20
grandchild 21
grandchild 22
grandchild 23
grandchild 24
grandchild 25
grandchild 26
grandchild 27
grandchild 28
grandchild 29
grandchild 30
grandchild 31
grandchild 32
grandchild 33
grandchild 34
grandchild 35
grandchild 36
grandchild 37
grandchild 38
grandchild 39
grandchild 40
grandchild 461
grandchild 462
grandchild 463
grandchild 464
grandchild 465
grandchild 466
grandchild 467
grandchild 468
grandchild 469
grandchild 470
grandchild 471
grandchild 472
grandchild 473
grandchild 474
grandchild 475
grandchild 476
grandchild 477
grandchild 478
grandchild 479
grandchild 480
grandchild 481
grandchild 482
grandchild 483
grandchild 484
grandchild 485
grandchild 486
grandchild 487
grandchild 488
grandchild 489
grandchild 490
grandchild 491
grandchild 492
grandchild 493
grandchild 494
grandchild 495
grandchild 496
grandchild 497
grandchild 498
grandchild 499
```

4、

```
(ygkldnherr@kali) - [~/Documents/os/exp3]
$ gcc -o fork3.out ./fork3.c
```

```

(ygkldnherr@kali)-[~/Documents/os/exp3]
$ ./fork3.out
^Cchild process1 is killed by parent!
child process2 is killed by parent!
parent process is killed!

(ygkldnherr@kali)-[~/Documents/os/exp3]
$ ./fork3.out
child process1 is killed by parent!
child process2 is killed by parent!
parent process is killed!

```

5、

```

(ygkldnherr@kali)-[~/Documents/os/exp3]
$ gcc -o fork4.out ./fork4.c

(ygkldnherr@kali)-[~/Documents/os/exp3]
$ ./fork4.out
^Cchild process 1 is killed!by parent
child process 2 is killed!by parent
parent process is killed

```

6、

```

(ygkldnherr@kali)-[~/Documents/os/exp3]
$ gcc -o fork5.out ./fork5.c

(ygkldnherr@kali)-[~/Documents/os/exp3]
$ ./fork5.out
child 1 process is sending message!
child 2 process is sending message!

```

问题讨论：

- 1、程序中的 `sleep(5)` 起什么作用？
`sleep(5)` 让子进程暂停 5 秒，子进程在向管道写入数据后，不立即退出，给父进程留出时间来 `wait()` 并 `read()` 同时也确保两个子进程不会几乎同时退出（从而避免输出乱序或资源竞争）。
- 2、子进程 1 和 2 为什么也能对管道进行操作？
 因为 `fork()` 复制了父进程的文件描述符表，管道在所有子进程中共享。

二、OS 源码阅读

1. 源代码文件名称
`Sched.h;fork.c`
2. 源代码功能描述
`Sched.h`

Includes:

head.h
fs.h
mm.h

```
File  
sched.h  
  
_SCHED_H =  
NR_TASKS = 64  
HZ = 100  
FIRST_TASK = task[0]  
LAST_TASK = task[NR_TASKS-1]  
TASK_RUNNING = 0  
TASK_INTERRUPTIBLE = 1  
TASK_UNINTERRUPTIBLE = 2  
TASK_ZOMBIE = 3  
TASK_STOPPED = 4  
INIT_TASK = { 0, 15, 15, 0, NULL,  
{(fn_ptr) 0}, 0, 0, 0, 0, 0,  
-1, 0, 0, 0, 0, 0, 0, 0, 0, 0,  
0, 0, 0, 0, ...  
CURRENT_TIME = (startup_  
time+jiffies/HZ)  
FIRST_TSS_ENTRY = 4  
FIRST_LDT_ENTRY = (FIRST_TSS_  
ENTRY + 1)  
_TSS = (((unsigned long)  
n)<<4)+(FIRST_TSS_ENTRY<<3))  
_LDT = (((unsigned long)  
n)<<4)+(FIRST_LDT_ENTRY<<3))  
ltr = __asm__ ("ltr %%eax:::\"a\"  
_TSS(n))  
lldt = __asm__ ("lldt %%eax:::  
\"a\" _LDT(n))  
str = __asm__ ("str %%eax  
%t\" \"subl %2,%%eax  
%t\" \"shrl $4,  
%%eax\" :\"a\" (n) :\"a\" (0),  
\"i\" (FIRST_TSS...  
switch_to = {struct {long a,  
b;} __tmp; __asm__ ("cmpl  
%%ecx, current  
%t\" \"je 1f  
%t\" \"xchgl %%ecx,...  
PAGE_ALIGN =  
(((n)+0xfff)&0xfff000)  
_set_base = __asm__ ("movw  
%%dx, %0  
%t\" \"rorl $16,%%edx  
%t\" \"movb %%dl, %1  
%t\" \"movb %%dh, %2\" :\"...  
_set_limit = __asm__ ("movw  
%%dx, %0  
%t\" \"rorl $16,%%edx  
%t\" \"movb %1, %%dh  
%t\" \"andb $0xf0, %%dh...  
set_base = _set_base( (char  
*)&(ldt) , base )  
set_limit = _set_limit(  
(char *)&(ldt) ,  
(limit-1)>12 )  
_get_base = ((unsigned long  
base; __asm__ ("movb %3, %%dh  
%t\" \"movb %2, %%dl  
%t\" \"shll $16, %%edx...  
get_base = _get_base( (char  
*)&(ldt) )  
get_limit = (( unsigned long  
_limit; __asm__ ("sll %1, %0  
%t\" \"incl %0\" :\"r\" ( _limit): \"r\"  
(segment...  
int copy_page_tables  
int free_page_tables  
void sched_init  
void schedule  
void trap_init  
volatile void panic  
int tty_write  
void sleep_on  
void interruptible_sleep_on  
void wake_up
```

整体功能： sched.h 是 Linux 0.01 的进程调度核心头文件，主要用于定义任务（task）的结构、状态、调度相关的宏与接口函数。它连接了任务结构体、任务切换、时间片、信号处理、TSS/LDT 管理、进程睡眠与唤醒机制等，是整个内核多任务机制的基础。作用可以总结为：定义任务控制块（task_struct）、任务状态、调度机制（schedule()）、任务切换宏（switch_to()）、时间控制（HZ/jiffies）、以及段描述符（TSS/LDT）管理接口。

基本参数定义：

```
#define NR_TASKS 64 //系统最多支持的任务数量（64 个）。
```

```
#define HZ 100 //系统时钟频率（100 次/秒），即每秒触发 100 次时钟中断，用于进程时间片调度。
```

任务状态定义：

```
#define TASK_RUNNING          0 //正在运行或可运行
```

```
#define TASK_INTERRUPTIBLE    1 //可被信号中断的睡眠
```

```
#define TASK_UNINTERRUPTIBLE  2 //不可被信号中断的睡眠
```

```
#define TASK_ZOMBIE           3 //僵尸状态（已退出但未被父进程回收）
```

```
#define TASK_STOPPED          4 //暂停
```

这几个宏定义了进程的五种状态

任务结构体 task_struct：

```
struct task_struct { ... };
```

这是进程控制块（PCB），记录进程的所有信息

初始任务定义 INIT_TASK：

```
#define INIT_TASK { ... }
```

这是系统启动时的第一个进程（即 task0），相当于 Linux 的 init_task

调度函数声明：

```
extern void schedule(void); //核心调度函数，根据优先级和时间片切换任务
```

```
extern void sleep_on(struct task_struct **p); //将当前进程置为睡眠状态（不可中断）
```

```
extern void interruptible_sleep_on(struct task_struct **p); //可被信号唤醒的睡眠
```

```
extern void wake_up(struct task_struct **p); //唤醒指定任务
```

段与任务描述符宏：

```
#define FIRST_TSS_ENTRY 4
```

```
#define _TSS(n) (((unsigned long)n)<<4)+(FIRST_TSS_ENTRY<<3))
```

```
#define ltr(n) __asm__ ("ltr %%ax:::a" (_TSS(n)))
```

这部分是段描述符（GDT/LDT/TSS）管理

任务切换宏 switch_to(n)：

这是整个 Linux 0.01 的核心切换宏，执行进程上下文切换：

 检查要切换的任务是否为当前任务；

 若不是，则：

 1 交换当前任务指针 current；

 2 用 ljmp 指令跳转到新任务的 TSS；

 3 若上一个任务使用了数学协处理器，则清除 TS 标志；

 4 最终完成寄存器、栈指针、段寄存器的切换。

地址操作宏：

```
#define set_base(ldt,base)
```

```
#define get_base(ldt)
```

这些是用于操作段描述符基址与限长的内联汇编函数。用于设置每个任务的用户态代码

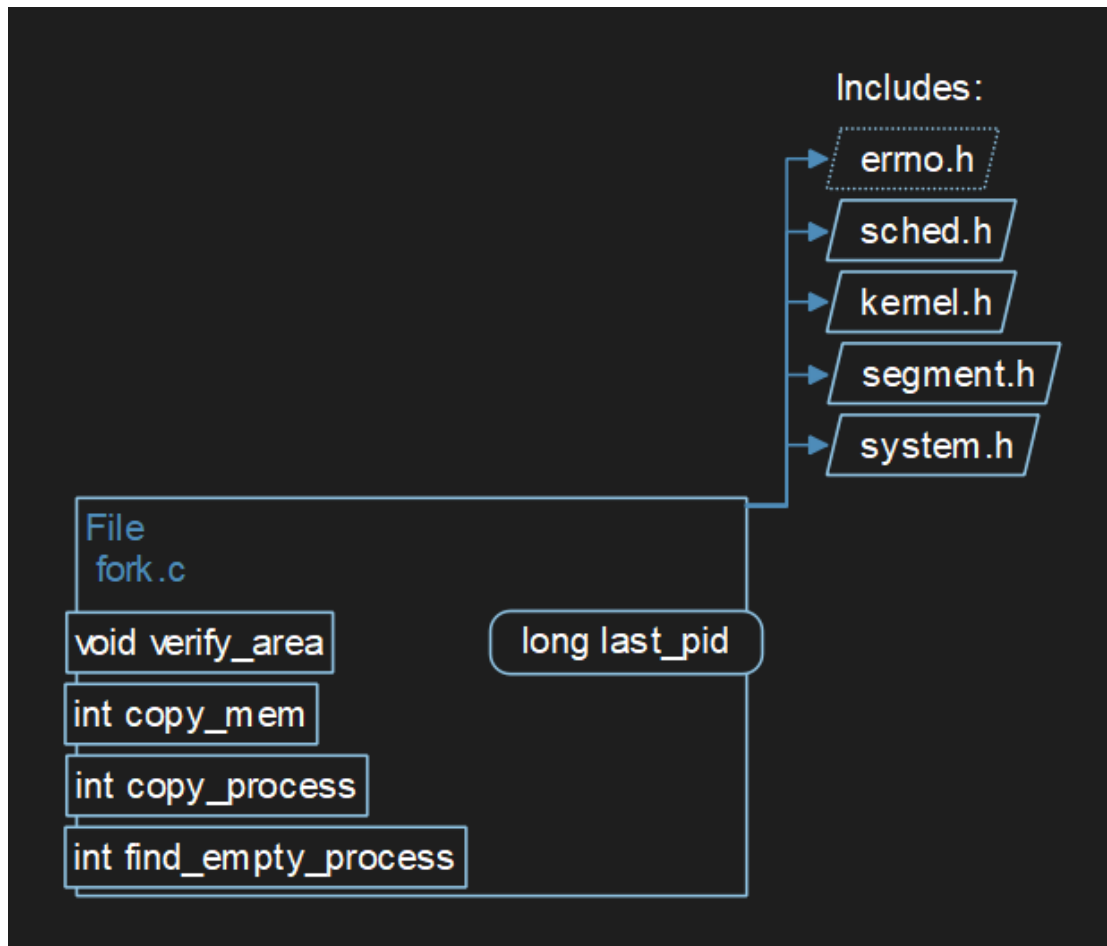
段和数据段的起始地址与长度。

时间控制宏：

`#define CURRENT_TIME (startup_time + jiffies / HZ)`

`jiffies` 是系统启动后经过的时钟中断次数。`CURRENT_TIME` 计算当前时间（单位秒），用于文件时间戳等。

fork.c



整体功能：

Linux 0.01 的 `fork.c` 文件实现了 系统调用 `fork()` 的核心逻辑，它完成的任务是：为当前进程创建一个几乎完全相同的新进程（子进程），包括复制寄存器状态、内存空间、文件表、TSS/LDT 结构等。

verify_area()

函数 1 输入：起始地址 `addr`，长度 `size`

函数 1 输出：无返回值

函数 1 功能：验证用户空间内存区域是否可写

copy_mem()

函数 2 输入：进程号 `nr`，任务指针 `p`

函数 2 输出：成功返回 0；失败返回 `-ENOMEM`

函数 2 功能：为子进程复制代码段/数据段页表

copy_process()

函数 3 输入：各种寄存器值（来自系统调用栈）

函数 3 输出：成功返回新 PID；失败返回负错误码

函数 3 功能：核心的进程复制函数

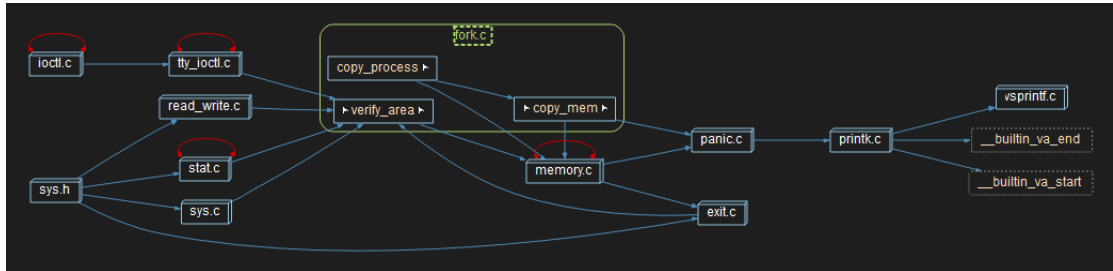
find_empty_process()

函数 4 输入：无

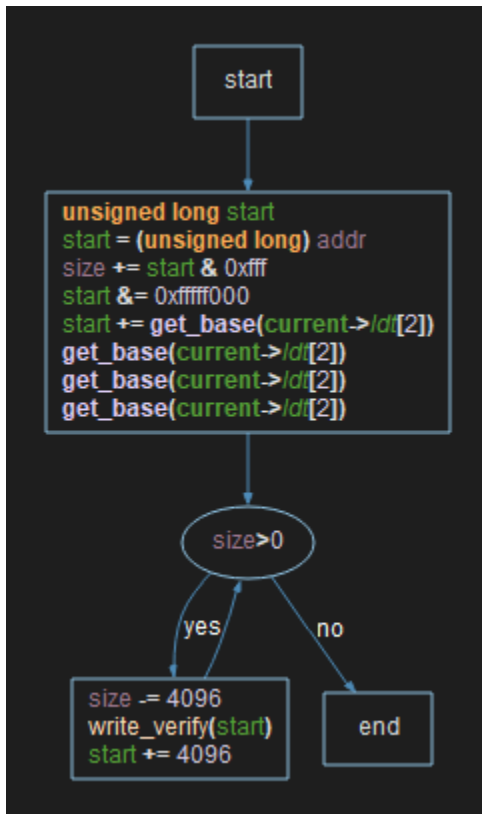
函数 4 输出：返回空闲任务号；失败返回 -EAGAIN

函数 4 功能：查找空闲任务槽并生成唯一 PID

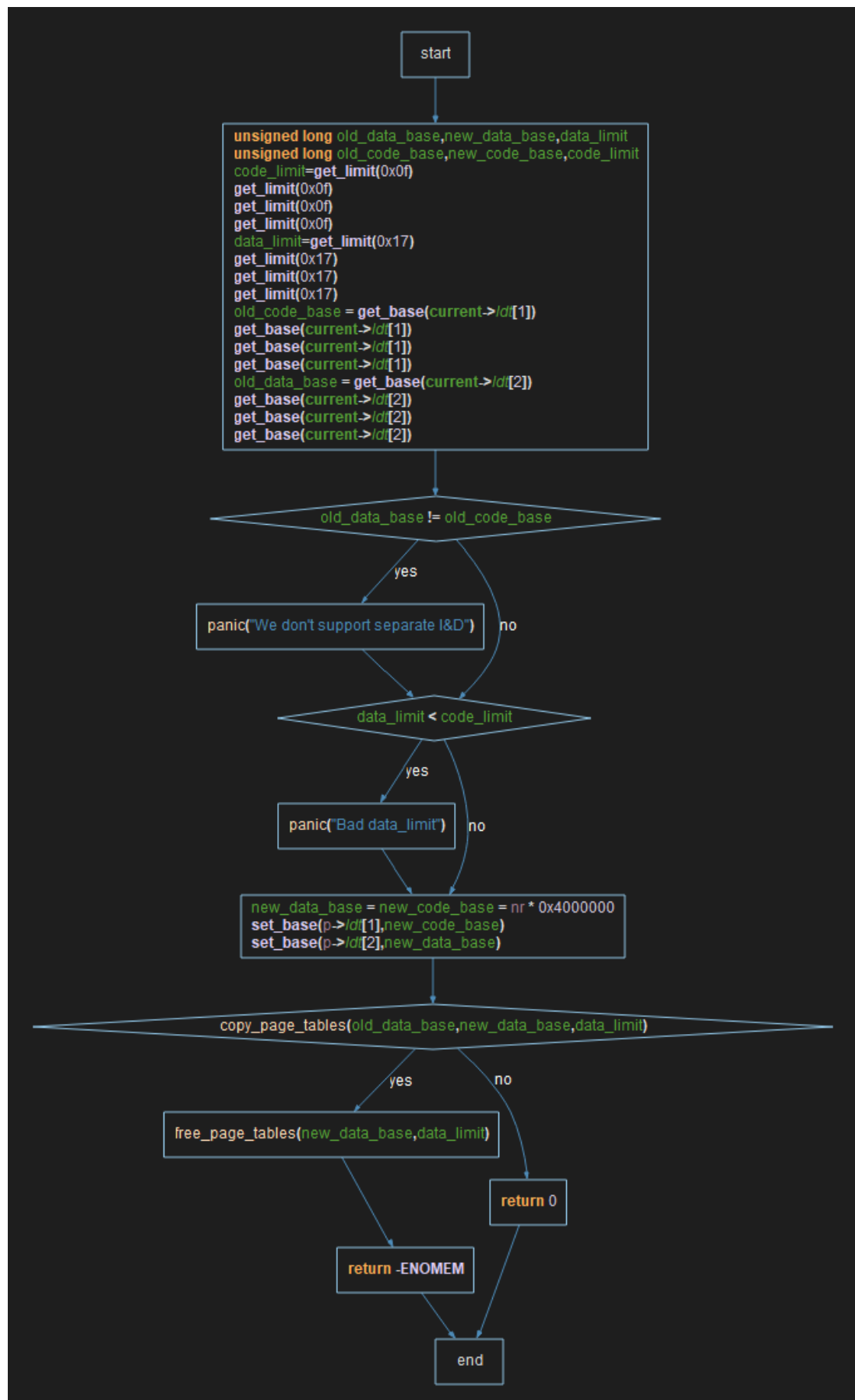
3. 程序流程图



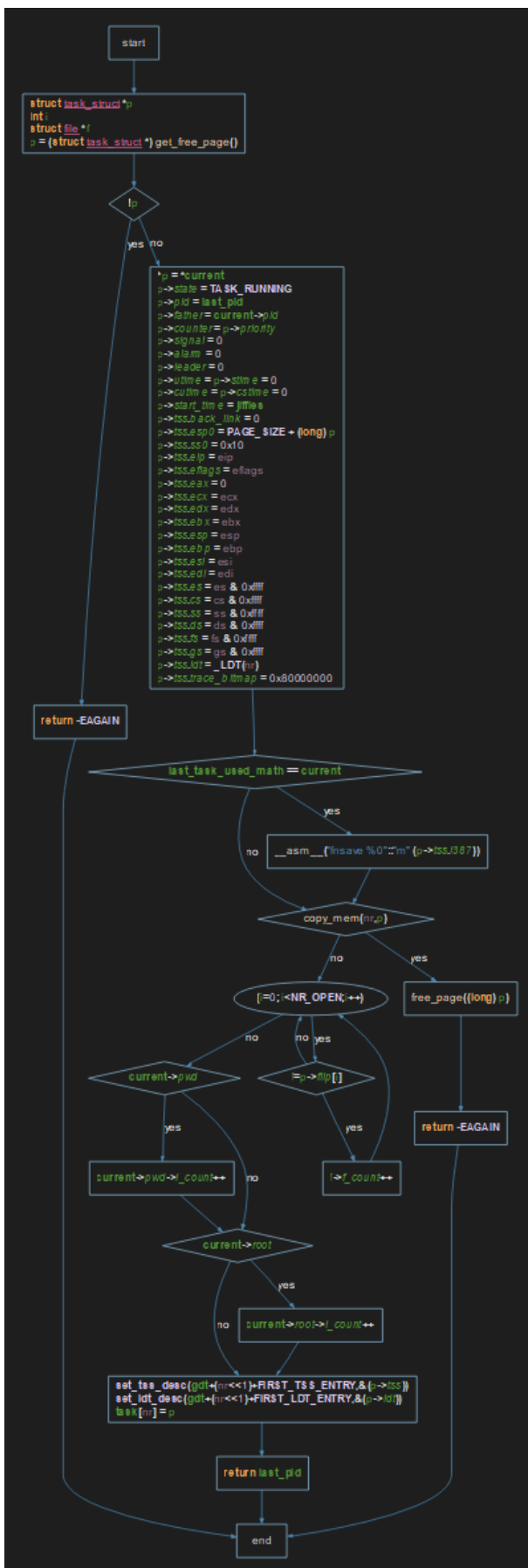
verify_area:



copy_mem:



copy_process:



find_empty:

